UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

GRADO EN FÍSICA

TRABAJO FIN DE GRADO

APLICACIÓN DE TENSOR NETWORKS EN MODELOS DE IA PARA CLASIFICACIÓN DE IMÁGENES

MANUEL ARENAS SÁNCHEZ

Dirigido por

ALEJANDRO MATA ALI

MARÍA FUENCISLA GILSANZ MUÑOZ

CURSO 2025-2026

TÍTULO: APLICACIÓN DE TENSOR NETWORKS EN MODELOS DE IA PARA CLASIFICACIÓN DE IMÁGENES

AUTOR: MANUEL ARENAS SÁNCHEZ

TITULACIÓN: GRADO EN FÍSICA

DIRECTOR/ES DEL PROYECTO: ALEJANDRO MATA ALI
MARÍA FUENCISLA GILSANZ MUÑOZ

FECHA: MARZO DE 2026

RESUMEN

ESTO SE ESCRIBE AL FINAL DEL PROYECTO

ABSTRACT

Porque se escribe esta parte en ingles??

Keywords: Palabras clave en inglés

AGRADECIMIENTOS

Se escribe al final del proyecto

Cita - frase célebre / Dedicatoria

Escribiré algo al final del proyecto

TABLA RESUMEN

[Rellena esta tabla, y borra la opción que no aplica. Si tu proyecto ha tenido tanto desarrollo de producto como investigación, haz constar "SI" en ambos casos]

	DATOS
Nombre y apellidos:	Manuel Arenas Sánchez
Título del proyecto:	Aplicacion de Tensor Networks en modelos de IA para clasificación de imágenes...
Directores del proyecto:	Alejandro Mata Ali y Maria Fuencisla Gilsanz
El proyecto se ha realizado en colaboración de una empresa o a petición de una empresa:	SI
El proyecto ha implementado un producto: (esta entrada se puede marcar junto a la siguiente)	NO
El proyecto ha consistido en el desarrollo de una investigación o innovación: (esta entrada se puede marcar junto a la anterior)	SI
Objetivo general del proyecto:	[...una frase...]

ÍNDICE

RESUMEN **4**

ABSTRACT **5**

AGRADECIMIENTOS **6**

TABLA RESUMEN **8**

1 INTRODUCCIÓN / ESTADO DEL ARTE **13**

1.1 Introducción a Deep Learning 13

1.2 Introducción a Tensor Networks 18

1.2.1 Concepto de tensor 18

1.2.2 Notación diagramática 20

1.2.3 Origen físico y papel del entrelazamiento 21

1.2.4 Descomposiciones tensoriales fundamentales 27

1.2.5 Topologías de Tensor Networks 30

1.3 Clasificación de imágenes con Tensor Networks 33

1.3.1 Codificación de los datos de entrada 33

1.3.2 Función de decisión y clasificación multiclase 34

1.3.3 Optimización del modelo: el algoritmo de barrido 35

1.3.4 Conexión con métodos kernel 36

2 OBJETIVOS **38**

2.1 Objetivos generales 38

2.2 Objetivos específicos 38

2.3 Objetivos que quedan fuera del proyecto 39

2.4 Beneficios del proyecto 39

3 DESARROLLO DEL TFG **41**

3.1 Planificación del trabajo fin de grado 41
3.2 Descripción de la solución, metodologías y herramientas empleadas 43
3.2.1 Recursos requeridos 43
3.3 Resultados del TFG 43

4 DISCUSIÓN 44

5 CONCLUSIONES 45
5.1 Conclusiones del trabajo 45
5.2 Conclusiones personales 45

6 FUTURAS LÍNEAS DE TRABAJO 46

REFERENCIAS 48

ANEXOS 49

ÍNDICE DE FIGURAS

1.1 Representación de las redes neuronales más comunes y simples. Empezando por el perceptrón, que fue el primer modelo de red neuronal propuesto en el año 1958 por Frank Rosenblatt, hasta llegar a modelos más modernos como las RNN. Fuente: [1]. 16

1.2 Ejemplo de representación diagramática para un tensor. Fuente: [2] 21

3.1 Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG. 41

ÍNDICE DE TABLAS

Todas las tablas incluidas en la memoria deben estar referenciadas para incluir en este índice.

1.1 Comparativa de las descomposiciones tensoriales fundamentales para un tensor de orden N con dimensiones $I_1 \times I_2 \times \cdots \times I_N$. 30

1.2 Comparativa de las principales topologías de Tensor Networks. N es el número de sitios, D la dimensión de enlace y d la dimensión local. 33

Capítulo 1. INTRODUCCIÓN / ESTADO DEL ARTE

1.1 Introducción a Deep Learning

El término *Deep Learning* nació de la necesidad de dar un nuevo nombre a las redes neuronales. No es más que una red neuronal con múltiples capas de transformación lineal entre la entrada y la salida. Con lo cual es importante entender antes el concepto de redes neuronales.

El origen de redes neuronales se remontan a la primera mitad del siglo XX, cuando los investigadores comenzaron a plantearse la posibilidad de modelar computacionalmente el funcionamiento del cerebro humano. Los primeros modelos fueron propuestos en 1943, pero estas arquitecturas carecían de capacidad de aprendizaje [3]. Pocos años después, en 1949 Hebb sentó las bases del aprendizaje en redes neuronales. Hebb postuló que, la conexión entre dos neuronas se refuerza cuando ambas se activan simultáneamente, concepto que hoy en día se conoce como la regla de Hebb.

La siguiente década trajo consigo el desarrollo de los primeros modelos entrenables mediante aprendizaje supervisado. Frank Rosenblatt presentó en 1958 el perceptrón, una unidad de procesamiento capaz de aprender a clasificar patrones linealmente separables a partir de ejemplos etiquetados [3]. Este dispositivo era, en esencia, una neurona artificial con un vector de pesos ajustables mediante un algoritmo de corrección del error, lo que lo convirtió en el primer modelo con capacidad de aprendizaje autónomo. No obstante, la euforia inicial se vio truncada en 1969, cuando Minsky y Papert demostraron matemáticamente que el perceptrón era incapaz de resolver problemas no linealmente separables, lo que provocó una notable desaceleración en la investigación sobre redes neuronales durante la década siguiente [3].

La recuperación del campo llegó de la mano de los modelos multicapa y del algoritmo de retropropagación del error. En 1986, Rumelhart, Hinton y Williams, demostraron que las redes de varias capas podían aprender representaciones internas útiles para resolver problemas de mayor complejidad [3]. Este hito abrió la puerta a las arquitecturas de redes neuronales más sofisticadas, que vendrían décadas después.

Una red neuronal es una estructura computacional formada por un conjunto de unidades de procesamiento simples, denominados neuronas. Éstas están interconectadas mediante pesos sinápticos ajustables, llamados en inglés *weights* y *biases*. Cada neurona recibe una serie de entradas, a las cuales llamaremos *inputs*; se calcula una suma ponderada de las mismas y se les aplican una función de activación –generalmente no lineal– para producir

una salida, la cual llamaremos *output*. El comportamiento de la red, queda determinado por los valores de estos pesos, que son modificados durante el entrenamiento con el objetivo de minimizar una función de error entre los *outputs* y las respuestas esperadas.

Formalizando esta descripción, dada una entrada $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$, la operación de una única neurona se expresa como:

$$y = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \tag{1.1}$$

donde $\mathbf{w} = (w_1, w_2, \ldots, w_n)$ es el vector de pesos sinápticos, b es el sesgo (*bias*), y $\sigma : \mathbb{R} \to \mathbb{R}$ es la función de activación. El argumento de σ es una transformación afín de la entrada: un producto escalar seguido de una traslación. La no linealidad introducida por σ es esencial, pues sin ella la composición de múltiples capas lineales se reduciría a una única transformación lineal, independientemente de la profundidad de la red [3].

Las funciones de activación más utilizadas en la práctica incluyen la sigmoide $\sigma(z) = 1/(1+e^{-z})$, la tangente hiperbólica $\sigma(z) = \tanh(z)$, y la unidad lineal rectificada (ReLU) $\sigma(z) = \text{máx}(0, z)$. La elección de la función de activación tiene consecuencias importantes sobre la dinámica del entrenamiento: funciones como la sigmoide pueden producir gradientes que se desvanecen en redes profundas, mientras que la ReLU mitiga parcialmente este problema [3].

Una capa completa de la red consiste en m neuronas que operan en paralelo sobre la misma entrada, produciendo un vector de salida $\mathbf{y} \in \mathbb{R}^m$:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1.2}$$

donde $\mathbf{W} \in \mathbb{R}^{m \times n}$ es la matriz de pesos de la capa y $\mathbf{b} \in \mathbb{R}^m$ el vector de sesgos. Una red neuronal profunda con L capas es la composición de L transformaciones de este tipo:

$$f(\mathbf{x}) = \sigma_L(\mathbf{W}_L \cdots \sigma_2(\mathbf{W}_2\, \sigma_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots + \mathbf{b}_L) \tag{1.3}$$

Los parámetros entrenables del modelo son el conjunto $\{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \ldots, \mathbf{W}_L, \mathbf{b}_L\}$, cuyo número total crece con la anchura y la profundidad de la red.

El entrenamiento de una red neuronal se formula como un problema de optimización. Dado un conjunto de entrenamiento $\{(\mathbf{x}_n, y_n)\}_{n=1}^{N_T}$, se define una *función de coste* (o función de pérdida) C que mide la discrepancia entre las predicciones del modelo $f(\mathbf{x}_n)$ y las etiquetas verdaderas y_n. El objetivo es encontrar los parámetros $\boldsymbol{\theta} = \{\mathbf{W}_l, \mathbf{b}_l\}$ que minimizan C:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}}\ C(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}}\ \frac{1}{N_T}\sum_{n=1}^{N_T} \mathcal{L}(f(\mathbf{x}_n; \boldsymbol{\theta}), y_n) \tag{1.4}$$

donde $\mathcal{L}$ es una función de pérdida por muestra (por ejemplo, el error cuadrático medio o la entropía cruzada). Este es un problema de minimización en un espacio de alta dimensión –las redes modernas tienen millones o miles de millones de parámetros–, que se resuelve mediante variantes del *descenso de gradiente* [3].

El algoritmo fundamental para calcular los gradientes necesarios es la *retropropagación* (*backpropagation*), que aplica sistemáticamente la regla de la cadena del cálculo diferencial para propagar el error desde la capa de salida hacia las capas inferiores. Para un parámetro θ cualquiera de la red, el gradiente se calcula como:

$$\frac{\partial C}{\partial \theta} = \frac{\partial C}{\partial \mathbf{y}_L} \cdot \frac{\partial \mathbf{y}_L}{\partial \mathbf{y}_{L-1}} \cdots \frac{\partial \mathbf{y}_{l+1}}{\partial \mathbf{y}_l} \cdot \frac{\partial \mathbf{y}_l}{\partial \theta} \tag{1.5}$$

donde $\mathbf{y}_l$ es la salida de la capa l-ésima. La actualización de los parámetros sigue entonces la regla $\theta \leftarrow \theta - \eta\, \partial C/\partial \theta$, donde $\eta > 0$ es la tasa de aprendizaje.

Desde la perspectiva de un físico, el entrenamiento de una red neuronal es conceptualmente análogo a la minimización de una función de energía en un sistema con muchos grados de libertad. La función de coste desempeña el papel del Hamiltoniano, los parámetros de la red son los grados de libertad, y el descenso de gradiente es un procedimiento análogo al enfriamiento simulado o a la relajación variacional. Esta analogía no es puramente retórica: tanto en física como en aprendizaje automático, el reto fundamental es explorar eficientemente un paisaje de alta dimensión en busca de mínimos globales (o, al menos, mínimos locales de buena calidad) [3].

Las redes neuronales se pueden clasificar según la topología de sus conexiones. Las redes *forward-propagation* (propagación hacia delante) o *feedforward* (FF), son aquellas en las que la información fluye unidireccionalmente desde la entrada hacia la salida sin ciclos, constituyen la forma más extendida. Dentro de este grupo, las redes convolucionales (CNN) destacan por su capacidad para procesar datos con estructura espacial mediante el uso de filtros que detectan patrones locales de forma eficiente [4]. Las redes recurrentes (RNN), por su parte, incorporan conexiones cíclicas que permiten mantener un estado interno a lo largo del tiempo, haciéndolas idóneas para el procesamiento de secuencias y datos temporales [4]. En términos de profundidad, las RNN pueden considerarse las más profundas de todas las arquitecturas, dado que sus cadenas causales se extienden de manera indefinida [3].

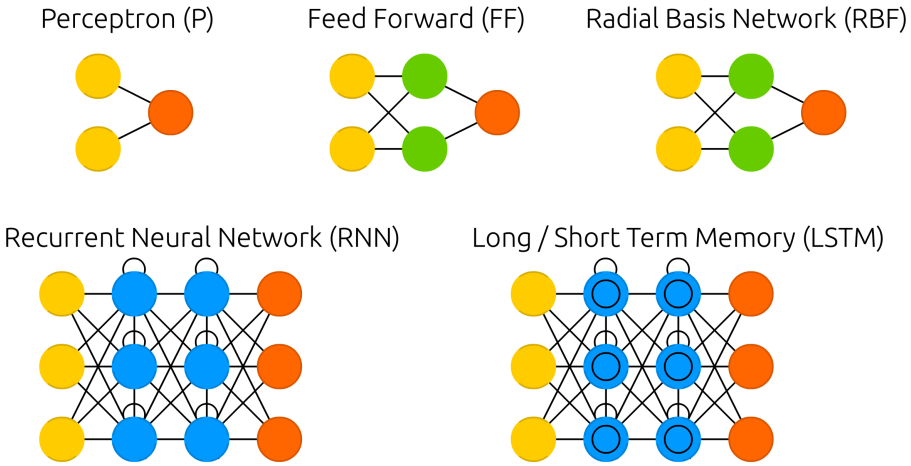

Figura 1.1: Representación de las redes neuronales más comunes y simples. Empezando por el perceptrón, que fue el primer modelo de red neuronal propuesto en el año 1958 por Frank Rosenblatt, hasta llegar a modelos más modernos como las RNN. Fuente: [1].

El campo del aprendizaje automático distingue también entre aprendizaje supervisado, no supervisado y por refuerzo. En el primero, la red dispone de etiquetas para cada muestra de entrenamiento. En el segundo, el objetivo es extraer estructura latente sin supervisión externa, como en los *autoencoders* o las máquinas de Boltzmann restringidas (RBM). El aprendizaje por refuerzo se basa en la interacción de un agente con un entorno, aprendiendo a seleccionar acciones que maximicen una señal de recompensa acumulada [3].

La diferencia fundamental entre las redes clásicas y las redes *deep* radica en su capacidad de representación. Una red con una única capa oculta puede, en teoría, aproximar cualquier función continua con la suficiente anchura —teorema de aproximación universal—, pero resulta ineficiente para capturar estructuras jerárquicas, como las que están presentes en las imágenes o en el lenguaje natural [5]. Las redes profundas, en cambio, aprenden representaciones en múltiples niveles de abstracción: las capas más bajas detectan características primitivas como bordes o frecuencias, mientras que las superiores combinan estas primitivas para reconocer objetos complejos o conceptos semánticos [4]. Esta propiedad de aprendizaje por representación hace que el *Deep Learning* sea especialmente adecuado para problemas en los que la ingeniería manual de características resultaría inviable.

Los modelos de *Deep Learning* exigen grandes volúmenes de datos de entrenamiento y una potencia de cómputo muy superior a la de los modelos clásicos, aspecto que durante décadas limitó su desarrollo. Sin embargo, la disponibilidad creciente de conjuntos de datos masivos y la proliferación de unidades de procesamiento gráfico (GPU) eliminaron estos cuellos de botella y permitieron el despegue definitivo del *Deep Learning* a partir de la segunda mitad de la década de 2000 [3].

En la actualidad, las redes neuronales de *Deep Learning* han revolucionado la inteligencia artificial al demostrar capacidades sin precedentes en la captura de patrones complejos

y representaciones jerárquicas [4]. En el campo de la visión por computador, la evolución de las arquitecturas CNN ha marcado hitos sucesivos en la clasificación de imágenes sobre ImageNet: desde AlexNet (2012), que inauguró la era de las CNN profundas en competición, hasta VGGNet, GoogLeNet y ResNet, cada una introduciendo innovaciones arquitectónicas que han impulsado progresivamente el estado del arte [4]. En biología estructural, AlphaFold2 supuso una revolución al predecir la estructura de prácticamente todas las proteínas conocidas con precisión atómica, reduciendo de años a días el tiempo necesario [4]. En procesamiento del lenguaje natural, la irrupción de los grandes modelos de lenguaje (LLM) basados en arquitecturas Transformer —ChatGPT, Llama, Claude 3 o DeepSeek— han demostrado capacidades que igualan o superan el rendimiento humano en amplias variedades de tareas [4].

Sin embargo, el éxito del Deep Learning no está exento de desafíos. Los modelos de última generación requieren cientos de miles de millones de parámetros, con los consiguientes costes computacionales y energéticos, así como dificultades para su despliegue en entornos con recursos limitados. Es en este contexto donde las Tensor Networks emergen como una alternativa prometedora, gracias a su capacidad para representar y comprimir eficientemente estructuras multilineales de alta dimensión, pueden reducir drásticamente el número de parámetros de los modelos de Deep Learning manteniendo su capacidad expresiva [4]. La intersección entre redes neuronales y redes tensoriales, lejos de ser una mera coincidencia formal, refleja una conexión profunda a través de sus estructuras matemáticas multilineales compartidas [4], abriendo el nuevo frente de investigación que este trabajo pretende explorar.

Conviene detenerse brevemente en esta cuestión, pues el paralelismo entre Deep Learning y física cuántica que motiva este trabajo se hace especialmente transparente cuando se examina el problema de la dimensionalidad. Esos elevados costes computacionales y energéticos son la consecuencia visible de una estrategia más profunda: la *sobreparametrización* característica del Deep Learning, en la que las redes modernas contienen típicamente muchos más parámetros que muestras de entrenamiento.

Bajo esta estrategia subyace una raíz matemática común a física y aprendizaje automático, conocida como la *maldición de la dimensionalidad*. Cuando se intenta representar una función general de N variables, cada una de las cuales puede tomar d valores, el número de parámetros necesarios para una representación exacta crece como d^N –de forma exponencial con el número de variables–. Una imagen modesta de 28×28 píxeles en escala de grises tiene $N = 784$ variables; representar la función de clasificación más general posible sobre estas imágenes requeriría un número de parámetros astronómicamente grande.

Este mismo problema aparece, con idéntica estructura matemática, en la física cuántica de muchos cuerpos. La función de onda de un sistema de N partículas cuánticas con dimensión local d es un tensor de d^N componentes, exactamente el mismo crecimiento exponencial. Como se desarrollará en la sección siguiente, las Tensor Networks nacieron precisamente para resolver este problema en física, explotando la estructura del entrelazamiento para representar la función de onda con un número polinomial de parámetros [5].

La convergencia de estos dos problemas –la sobreparametrización en Deep Learning y la explosión exponencial en mecánica cuántica– no es una mera coincidencia formal. En ambos casos, la solución reside en explotar la estructura interna del objeto que se desea representar: en física, la estructura de entrelazamiento de los estados cuánticos; en aprendizaje automático, la estructura de correlaciones de los datos. Las Tensor Networks proporcionan un marco unificado para abordar ambos problemas, y su aplicación a modelos de clasificación de imágenes –objetivo central de este trabajo– puede interpretarse como la transferencia de herramientas desarrolladas durante décadas en física de la materia condensada al dominio del aprendizaje automático [4].

1.2 Introducción a Tensor Networks

Antes de introducir o dar una definición de lo que es una *Tensor Network* (TN), es conveniente aclarar primero la definición de un tensor.

1.2.1 Concepto de tensor

Un tensor lo podemos ver definido de diferentes formas dependiendo del campo en el que estemos trabajando.

Desde un puesto de vista computacional, los tensores son *arrays* multidimensionales que generalizan de forma natural los conceptos de escalar, vector y matriz a dimensiones arbitrariamente superiores. Desde un punto de vista físico-matemático, los tensor son objetos matemáticos de orden superior, que representan relaciones multidimensionales entre distintas variables o grados de libertad. Dicho de otra forma, un tensor de orden N es un objeto cuyos elementos tienen N índices y están asociados a N vectores base.

$$R^{I_1 \times I_2 \times \cdots \times I_N}$$

donde N es el número de modos y las longitudes $I_1, \ldots, I_N$ representan las dimensiones de cada modo. [4]

Bajo esta definición, los escalares son tensores de orden cero, los vectores son tensores de orden 1, y las matrices son tensores de orden 2; todo caso superior constituye propiamente lo que en la literatura se denomina, tensor de alto orden. En la notación convencional, se distinguen los diferentes objetos según su orden: las letras minúsculas en cursiva (a) denotan escalares, las letras minúsculas en negrita ($\mathbf{a}$) denotan vectores, las letras mayúsculas en negrita ($\mathbf{A}$) denotan matrices, y las letras mayúsculas en tipografía cursiva en negrita ($\boldsymbol{\mathcal{A}}$) denotan tensores de orden superior. [4]

La intuición física detrás de los tensores resulta natural cuando se considera, que muchos fenómenos del mundo real son inherentemente multidimensionales. Una imagen en color, por ejemplo, es un tensor de orden tres: altura, anchura y canales cromáticos. Los datos de resonancia magnética funcional (fMRI) constituyen tensores de orden cuatro, compuestos

por vóxeles tridimensionales que varían en el tiempo. En mecánica cuántica, las funciones de onda variacionales de sistemas de muchos cuerpos son, igualmente, tensores de alto orden [4].

El concepto de tensor tiene raíces en el siglo XIX, ligado al desarrollo del cálculo diferencial absoluto por Ricci-Curbastro y Levi-Civita, aunque fue la formulación de la relatividad general por Einstein en 1915 la que consagró los tensores como herramienta fundamental de la física matemática. La notación de índices de Einstein, que convierte la suma sobre índices repetidos en una convención implícita, es todavía hoy estándar en muchos contextos físicos.

$$\boldsymbol{\mathcal{T}} = \sum_{i_1,i_2,\ldots,i_n} T_{i_1 i_2 \ldots i_n} \hat{b_{i_1}} \hat{b_{i_2}} \ldots \hat{b_{i_n}} \tag{1.6}$$

La operación central en el álgebra tensorial es la contracción tensorial. Dados dos tensores $\boldsymbol{\mathcal{T}}$ y $\boldsymbol{\mathcal{R}}$, de orden n y m respectivamente, con elementos $T_{i_1 i_2 \ldots i_n}$ y $R_{j_1 j_2 \ldots j_m}$ y en la misma base ortonormal $\hat{b}_i$, diremos que el tensor $\boldsymbol{\mathcal{C}}$ es la contracción de ambos tensores por sus último q índices cuando:

$$\mathcal{C} = \sum_{i_1,i_2,\ldots,i_{n-q},j_1,j_2,\ldots,j_{m-q}} C_{i_1 i_2 \ldots i_{n-q}, j_1 j_2 \ldots j_{m-q}} \hat{b}_{i_1} \hat{b}_{i_2} \ldots \hat{b}_{n-q} \hat{b}_{j_1} \hat{b}_{j_2} \ldots \hat{b}_{m-q} \tag{1.7}$$

siendo sus elementos

$$C_{i_1 i_2 \ldots i_{n-q}, j_1 j_2 \ldots j_{m-q}} = \sum_{k_1,k_2,\ldots,k_q} T_{i_1 i_2 \ldots i_{n-q}, k_1 k_2 \ldots k_q} R_{j_1 j_2 \ldots j_{m-q}, k_1 k_2 \ldots k_q} \tag{1.8}$$

De esta definición podemos sacar que la contracción tensorial es la operación mediante la cual dos tensores se contraen en un único tensor a lo largo de sus pares de índices asociados. Existen formas diferentes de contracciones, algunas implican más de dos tensores o un índice repetido en más de dos tensores o incluso repetidos en el mismo tensor. El caso más familiar de contracción tensorial es la multiplicación matricial:

$$\mathbf{AB} = \sum_k A_{ik} B_{kj} \hat{b}_i \hat{b}_j$$

o el producto escalar entre dos vectores:

$$\vec{v} \cdot \vec{w} = \sum_i v_i w_i$$

Entre las operaciones tensoriales fundamentales cabe destacar la traza de una matriz:

$$Tr(A) = \sum_i A_{ii} \tag{1.9}$$

Esto sería la contracción de una matriz consigo misma a través de sus índices repetidos. Debido a esta operación podemos ver que, un tensor también sirve como mapeo entre dos espacios vectoriales. Esto es un tensor, que recibe otro tensor y devuelve un tercer tensor. Por cada índice contraído estamos mezclando componentes del tensor de entrada, reduciendo en 1 el orden del tensor de salida. Por otro lado, los índices que no han sido contraídos aumentarán en 1 el orden del tensor de salida.

1.2.2 Notación diagramática

La representación algebraica de las contracciones tensoriales, aunque precisa, se vuelve rápidamente inmanejable a medida que aumenta el número de tensores y la complejidad de sus conexiones. Fue Roger Penrose [6] quien, a principios de la década de 1970, introdujo la notación diagramática para tensores, proporcionando una alternativa visual que transformó la manera en que se razona sobre estas estructuras. Los diagramas de redes tensoriales son hoy ampliamente utilizados para describir algoritmos cuánticos y algoritmos de aprendizaje automático [4].

La idea central de la notación de Penrose es representar cada tensor como un nodo del que emanan tantas aristas como índices tenga el tensor. Concretamente: un nodo con una sola arista representa un vector $\mathbf{a} \in \mathbb{R}^I$; un nodo con dos aristas representa una matriz $\mathbf{A} \in \mathbb{R}^{I \times J}$; y un nodo con tres aristas representa un tensor $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$. Para indicar que un par de índices tensoriales está siendo contraído, ambas aristas se unen mediante una línea. Por ejemplo, la contracción de un tensor de orden uno con uno de orden dos es la multiplicación matriz-vector habitual. Las aristas que no se conectan a ningún otro tensor se denominan aristas libres, y el número de estas aristas en el diagrama resultante es precisamente el orden del tensor final. Dos nodos que no están unidos mediante aristas, representan el producto tensorial entre ellos.

La notación gráfica tensorial ofrece numerosas ventajas sobre la notación tradicional: en forma gráfica, los índices no necesitan nombres ni etiquetas; operaciones como el producto escalar, la traza tensorial y la contracción pueden expresarse sin notación adicional; y el orden del tensor resultante puede leerse simplemente contando el número de aristas libres. En particular, si en el diagrama no quedan aristas libres tras todas las contracciones, el resultado es necesariamente un escalar. Esta propiedad convierte los diagramas en una herramienta de verificación inmediata de la consistencia dimensional de operaciones complejas [7]. Sobre esta base visual puede introducirse ya, de manera natural, el concepto de Tensor Network.

Una Tensor Network (TN) es, en esencia, una representación gráfica de una ecuación multilineal entre diferentes tensores. Esta representación se apoya en la notación diagramática de Penrose, que con el tiempo se ha ido adaptando a las necesidades de disciplinas como la física, las matemáticas o la ciencia computacional [8]. Puede verse un ejemplo en la Figura 1.2, donde aparece la Tensor Network que representa el 2-tensor T_{ip} cuyos elementos se obtienen mediante la operación de contracción

$$T_{ip} = \sum_{j,k,l,m,n,o} A_{ijkl} B_{jm} C_{kmn} D_{lo} E_{nop} \tag{1.10}$$

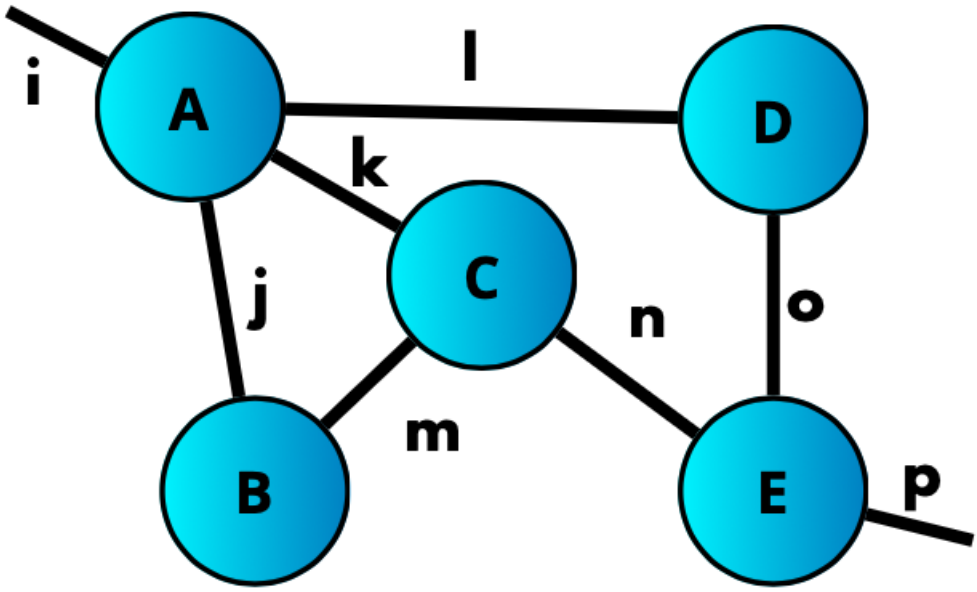

Figura 1.2: Ejemplo de representación diagramática para un tensor. Fuente: [2]

La motivación fundamental para el uso de las Tensor Networks es de naturaleza computacional. Un tensor de orden N con dimensión d en cada nodo tiene d^N componentes, un número que crece exponencialmente con el orden del tensor, haciendo que su almacenamiento y manipulación directos sean computacionalmente prohibitivos; este fenómeno se conoce como la *maldición de la dimensionalidad*. Las Tensor Networks abordan este problema representando el tensor original no de forma explícita, sino como el resultado de contraer un conjunto de tensores de menor orden, cada uno con un número de parámetros manejable, de modo que la complejidad resultante escala de forma polinomial en lugar de exponencial [4, 7]. Esta idea general se entiende mejor cuando se examina el contexto físico en el que surgió.

1.2.3 Origen físico y papel del entrelazamiento

Las Tensor Networks no nacieron como una herramienta matemática abstracta ni como un recurso computacional genérico: su origen está profundamente enraizado en la física cuántica, concretamente en el problema de describir sistemas de muchas partículas interactuantes. Comprender este origen es esencial para apreciar por qué las Tensor Networks resultan tan eficaces cuando se aplican fuera de su contexto original, como en el aprendizaje automático.

El problema de muchos cuerpos en mecánica cuántica.

En mecánica cuántica, el estado de un sistema físico queda completamente descrito por su

función de onda $|\psi\rangle$, que pertenece a un espacio de Hilbert $\mathcal{H}$. Para una única partícula con espín 1/2, por ejemplo, el espacio de Hilbert es de dimensión $d = 2$: la partícula puede encontrarse en el estado $|0\rangle$ (espín arriba) o $|1\rangle$ (espín abajo), o en cualquier superposición de ambos. Hasta aquí, el problema es perfectamente manejable.

La dificultad surge cuando consideramos un sistema compuesto por N partículas. En mecánica cuántica, el espacio de Hilbert del sistema total es el producto tensorial de los espacios individuales:

$$\mathcal{H}_{\text{total}} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_N \tag{1.11}$$

Si cada partícula tiene una dimensión local d (por ejemplo, $d = 2$ para espines 1/2), la dimensión del espacio total es d^N. Un estado general del sistema se escribe como:

$$|\psi\rangle = \sum_{s_1,s_2,\ldots,s_N} C_{s_1 s_2 \ldots s_N} |s_1\rangle \otimes |s_2\rangle \otimes \cdots \otimes |s_N\rangle \tag{1.12}$$

donde el tensor de coeficientes $C_{s_1 s_2 \ldots s_N}$ contiene d^N números complejos que describen completamente el estado cuántico [4]. Este tensor es exactamente un tensor de orden N en el sentido definido en la sección anterior.

El crecimiento exponencial de este espacio constituye el obstáculo fundamental de la física cuántica de muchos cuerpos. Para un sistema modesto de $N = 40$ espines 1/2, el tensor de coeficientes tiene $2^{40} \approx 10^{12}$ componentes complejas. Almacenar este tensor en memoria requeriría del orden de 16 terabytes (a 16 bytes por número complejo de doble precisión). Para $N = 100$, la cifra asciende a $2^{100} \approx 10^{30}$, un número que excede con creces la capacidad de cualquier computador existente o concebible [5]. Aparece así, de nuevo, la *maldición de la dimensionalidad*, ahora vista desde su contexto físico original.

La estructura de los estados cuánticos físicamente relevantes.

A pesar de la enormidad del espacio de Hilbert, la naturaleza ofrece una salida: los estados cuánticos que aparecen en la práctica –estados fundamentales de Hamiltonianos con interacciones locales, estados térmicos a baja temperatura, o estados que resultan de evoluciones temporales físicamente razonables– no ocupan la totalidad del espacio d^N-dimensional, sino que se concentran en una región mucho más pequeña [7].

La razón de esta restricción reside en la estructura del entrelazamiento cuántico. El entrelazamiento es una propiedad genuinamente cuántica que describe las correlaciones no clásicas entre subsistemas. Cuando un sistema de muchas partículas se encuentra en su estado de mínima energía (estado fundamental), el grado de entrelazamiento entre una región del sistema y el resto está limitado por una ley conocida como la *ley de área* (*area law*): la entropía de entrelazamiento entre un subsistema y su complemento no crece con el volumen del subsistema, sino con el tamaño de la frontera que los separa [9]. En un sistema unidimensional –como una cadena de espines–, la frontera entre cualquier bipar-

tición es simplemente un punto, lo que implica que el entrelazamiento permanece acotado por una constante independientemente del tamaño del sistema.

Esta observación tiene una consecuencia profunda: si el entrelazamiento está acotado, el tensor de coeficientes $C_{s_1 s_2 \ldots s_N}$ posee una estructura interna muy particular que permite representarlo de forma compacta, sin necesidad de almacenar explícitamente sus d^N componentes. Las Tensor Networks son precisamente la herramienta matemática que explota esta estructura.

Para entender por qué esta compresión es posible, conviene introducir con algo más de detalle el concepto de entrelazamiento cuántico, que determina qué estados pueden describirse eficientemente mediante una Tensor Network.

Estados producto y estados entrelazados.

Para comprender el entrelazamiento, consideremos primero su opuesto: un estado sin entrelazamiento, denominado *estado producto*. Si un sistema de N partículas se encuentra en un estado producto, su función de onda se puede escribir como:

$$|\psi_{\text{prod}}\rangle = |\phi_1\rangle \otimes |\phi_2\rangle \otimes \cdots \otimes |\phi_N\rangle \tag{1.13}$$

donde cada $|\phi_k\rangle$ es el estado individual de la partícula k-ésima. En un estado producto, cada partícula está completamente descrita por su estado individual: el resultado de una medida sobre la partícula k es estadísticamente independiente de cualquier medida sobre las demás. El tensor de coeficientes $C_{s_1 s_2 \ldots s_N}$ de un estado producto es simplemente el producto exterior de N vectores, lo que significa que solo tiene rango 1 y puede almacenarse con $N \cdot d$ parámetros en lugar de d^N.

Sin embargo, la inmensa mayoría de los estados cuánticos no son estados producto. En general, el estado del sistema presenta correlaciones cuánticas entre sus partes que no pueden explicarse de forma clásica. Este fenómeno recibe el nombre de *entrelazamiento cuántico*. En un estado entrelazado, la medida sobre un subsistema afecta instantáneamente a la descripción del resto del sistema, independientemente de la distancia física que los separe.

Descomposición de Schmidt y entropía de entrelazamiento.

Para cuantificar el entrelazamiento, se utiliza la *descomposición de Schmidt*. Consideremos una bipartición del sistema en dos subsistemas A y B (por ejemplo, las primeras n partículas y las últimas $N-n$). Cualquier estado puro $|\psi\rangle$ admite una descomposición de la forma:

$$|\psi\rangle = \sum_{\alpha=1}^{\chi} \lambda_\alpha \left|\phi_\alpha^A\right\rangle \otimes \left|\phi_\alpha^B\right\rangle \tag{1.14}$$

donde $\{\left|\phi_\alpha^A\right\rangle\}$ y $\{\left|\phi_\alpha^B\right\rangle\}$ son bases ortonormales de los subsistemas A y B respectivamente, los coeficientes $\lambda_\alpha \geq 0$ se denominan *valores de Schmidt* y satisfacen $\sum_\alpha \lambda_\alpha^2 = 1$, y χ es el

rango de Schmidt, es decir, el número de valores de Schmidt no nulos.

Un estado producto tiene $\chi = 1$ (un único valor de Schmidt $\lambda_1 = 1$), mientras que un estado máximamente entrelazado tiene $\chi = \text{mín}(d^n, d^{N-n})$ con todos los valores de Schmidt iguales. La descomposición de Schmidt está íntimamente relacionada con la descomposición en valores singulares (SVD) del álgebra lineal: si reorganizamos el tensor de coeficientes $C_{s_1 \ldots s_N}$ como una matriz M de dimensiones $d^n \times d^{N-n}$, la SVD de M proporciona directamente los valores de Schmidt y las bases de Schmidt [10].

La medida estándar del entrelazamiento para estados puros es la *entropía de entrelazamiento de von Neumann*, definida a partir de la matriz densidad reducida del subsistema A:

$$\rho_A = \text{Tr}_B \left(|\psi\rangle \langle\psi| \right) = \sum_{\alpha=1}^{\chi} \lambda_\alpha^2 \left| \phi_\alpha^A \right\rangle \left\langle \phi_\alpha^A \right| \tag{1.15}$$

$$S(\rho_A) = -\text{Tr} \left(\rho_A \log \rho_A \right) = -\sum_{\alpha=1}^{\chi} \lambda_\alpha^2 \log \lambda_\alpha^2 \tag{1.16}$$

La entropía $S(\rho_A)$ es cero si y solo si el estado es un producto (no hay entrelazamiento), y alcanza su valor máximo $\log\Big(\text{mín}(d^n, d^{N-n})\Big)$ para un estado máximamente entrelazado. Cuanto mayor es la entropía de entrelazamiento, más compleja es la estructura de correlaciones entre los subsistemas y, en consecuencia, más difícil resulta representar el estado de forma compacta [4].

La ley de área del entrelazamiento.

El resultado clave que conecta el entrelazamiento con las Tensor Networks es la llamada *ley de área* (*area law*). Para sistemas cuánticos en los que las interacciones entre partículas son locales –es decir, cada partícula interactúa solo con sus vecinas más próximas–, los estados de baja energía (y en particular el estado fundamental) satisfacen una propiedad notable: la entropía de entrelazamiento entre una región A y el resto del sistema B no escala con el volumen de A, sino con el tamaño de la frontera que separa A de B [9].

En un sistema unidimensional (una cadena de espines), la frontera entre cualquier bipartición contigua es un único punto, independientemente del tamaño del sistema. Esto implica que la entropía de entrelazamiento está acotada por una constante:

$$S(\rho_A) \leq \text{const.} \quad \text{(sistemas 1D con gap espectral)} \tag{1.17}$$

En un sistema bidimensional, la frontera es una línea cuya longitud L es proporcional al perímetro de la región A, y la entropía crece proporcionalmente a L en lugar de al área de la región [9].

La ley de área tiene una consecuencia computacional directa y de gran alcance. Si el entrelazamiento está acotado, el rango de Schmidt χ efectivo en cualquier bipartición

del sistema es finito (o crece lentamente), lo que significa que la función de onda puede representarse fielmente mediante un número de parámetros que crece solo polinomialmente con el tamaño del sistema N, en lugar de exponencialmente. Las Tensor Networks son la herramienta que materializa esta compresión [7].

Consecuencias para las Tensor Networks.

La relación entre entrelazamiento y Tensor Networks se concreta de la siguiente manera. Un MPS con dimensión de enlace D puede representar estados cuya entropía de entrelazamiento satisface $S \leq \log D$ para cualquier bipartición contigua. Puesto que la ley de área garantiza que el estado fundamental de una cadena unidimensional con gap espectral tiene $S \leq$ const., basta con elegir un D finito para obtener una representación fiel del estado, con un número total de parámetros del orden de $N \cdot d \cdot D^2$, que es polinomial en N [10].

En este sentido, la dimensión de enlace D adquiere una interpretación física muy clara: controla la cantidad máxima de entrelazamiento que la red tensorial puede albergar. Si D es demasiado pequeño, la TN no puede capturar todas las correlaciones del estado y la representación será aproximada; si D es suficientemente grande, la representación es exacta. Esto explica por qué D es el hiperparámetro central de los modelos basados en Tensor Networks, tanto en física cuántica como en aprendizaje automático [7, 4].

Es importante señalar que no todos los estados satisfacen la ley de área. Los sistemas cuánticos en puntos críticos –donde se produce una transición de fase cuántica– exhiben correcciones logarítmicas a la ley de área, del tipo $S \sim \log N$, lo que requiere dimensiones de enlace mayores o arquitecturas tensoriales más sofisticadas como MERA [11]. Los estados completamente aleatorios, por su parte, violan la ley de área de forma masiva, con una entropía que escala con el volumen del subsistema; estos estados no pueden representarse eficientemente mediante ninguna Tensor Network, pero tampoco son físicamente relevantes.

La ley de área no es exclusiva de la mecánica cuántica. En el contexto del aprendizaje automático, la estructura de correlaciones de los datos de entrada juega un papel análogo al del entrelazamiento: los píxeles vecinos en una imagen están más correlacionados que los lejanos, lo que impone una estructura de correlación local que las Tensor Networks pueden explotar eficientemente [7]. Esta analogía entre la localidad espacial en imágenes y la ley de área en física cuántica es una de las razones por las que los modelos tensoriales, originalmente diseñados para la física, resultan naturalmente adecuados para tareas de clasificación de imágenes.

El DMRG y el nacimiento de las Tensor Networks modernas.

Sobre esta base física se entiende mejor el punto de inflexión histórico del campo, que llegó en 1992, cuando Steven R. White propuso el algoritmo *Density Matrix Renormalization Group* (DMRG) [12]. El DMRG fue diseñado para calcular el estado fundamental de cadenas cuánticas unidimensionales y revolucionó el campo de la materia condensada al proporcionar resultados de una precisión sin precedentes para sistemas que hasta entonces eran intratables.

Aunque White formuló el DMRG como un procedimiento iterativo de truncamiento del espacio de Hilbert basado en la matriz densidad reducida, posteriormente se demostró que el estado variacional que el DMRG optimiza es, de hecho, un *Matrix Product State* (MPS) [10]. Es decir, el DMRG puede entenderse como un algoritmo de optimización variacional sobre el espacio de tensores en formato MPS. Este descubrimiento fue fundamental porque conectó un algoritmo numérico exitoso con un marco teórico riguroso: la teoría de Tensor Networks.

A partir de esta conexión, se desarrollaron nuevas arquitecturas tensoriales adaptadas a diferentes geometrías y tipos de correlaciones. Los *Projected Entangled Pair States* (PEPS) generalizan el MPS a dos dimensiones [13]; los *Tree Tensor Networks* (TTN) adoptan una estructura jerárquica en árbol; y la *Multi-scale Entanglement Renormalization Ansatz* (MERA) introduce una estructura de capas diseñada para capturar correlaciones a múltiples escalas en sistemas con invariancia de escala, como los sistemas críticos [11]. Cada una de estas arquitecturas será tratada en detalle más adelante.

De la física cuántica al aprendizaje automático.

La conexión entre Tensor Networks y aprendizaje automático no es una mera analogía superficial, sino que se fundamenta en una correspondencia estructural profunda. En ambos campos, el problema central es el mismo: representar y manipular objetos que viven en espacios de dimensionalidad exponencialmente grande [4].

En mecánica cuántica, el objeto es la función de onda $|\psi\rangle$, un tensor de orden N con d^N componentes que describe las correlaciones entre N partículas. En aprendizaje automático, el objeto análogo es el tensor de pesos W que parametriza un modelo de clasificación; cuando se utiliza un mapa de características (*feature map*) basado en productos tensoriales, este tensor de pesos también puede tener un número exponencial de componentes [7].

La idea clave que permite trasladar las Tensor Networks de la física al aprendizaje automático es la siguiente: del mismo modo que los estados físicos relevantes no ocupan todo el espacio de Hilbert debido a las restricciones impuestas por el entrelazamiento, los modelos de clasificación eficaces no necesitan explorar la totalidad del espacio exponencial de pesos. La estructura de correlaciones presente en los datos de entrenamiento –por ejemplo, la localidad espacial en imágenes– desempeña un papel análogo al de la ley de área en física cuántica [7]. Las Tensor Networks explotan esta estructura para representar el tensor de pesos de forma compacta, con un número de parámetros que crece polinomialmente en lugar de exponencialmente con el tamaño de la entrada.

Esta correspondencia no se limita al nivel conceptual. Los algoritmos de optimización desarrollados para entrenar modelos tensoriales en aprendizaje automático son descendientes directos del DMRG y sus variantes; la dimensión de enlace que controla la precisión de un MPS en física cuántica es exactamente el mismo hiperparámetro que controla la capacidad expresiva de un clasificador tensorial [7, 4]. Este paralelismo íntimo entre los dos campos constituye la motivación central del presente trabajo.

1.2.4 Descomposiciones tensoriales fundamentales

En las secciones anteriores se ha establecido que las Tensor Networks permiten representar tensores de alto orden de forma compacta, explotando la estructura de entrelazamiento o correlación presente en los datos. Sin embargo, para comprender cómo se construyen estas representaciones en la práctica, es necesario conocer las descomposiciones tensoriales que constituyen sus fundamentos matemáticos. Estas descomposiciones generalizan al caso tensorial técnicas de factorización matricial bien conocidas en álgebra lineal, como la descomposición en valores singulares (SVD).

Descomposición en valores singulares (SVD).

La descomposición en valores singulares es la herramienta algebraica central sobre la que se construyen la mayoría de las descomposiciones tensoriales. Dada una matriz $\mathbf{A} \in \mathbb{R}^{M\times N}$, la SVD la factoriza como:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \tag{1.18}$$

donde $\mathbf{U} \in \mathbb{R}^{M\times M}$ y $\mathbf{V} \in \mathbb{R}^{N\times N}$ son matrices ortogonales (sus columnas forman bases ortonormales), y $\mathbf{\Sigma} \in \mathbb{R}^{M\times N}$ es una matriz diagonal cuyos elementos $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ son los *valores singulares* de $\mathbf{A}$. El número de valores singulares no nulos es el rango de la matriz.

La propiedad clave de la SVD para las Tensor Networks es que proporciona la mejor aproximación de rango bajo en norma de Frobenius. Si truncamos la SVD conservando únicamente los R valores singulares más grandes, obtenemos la matriz $\tilde{\mathbf{A}} = \mathbf{U}_R\mathbf{\Sigma}_R\mathbf{V}_R^\top$ que minimiza $\|\mathbf{A} - \tilde{\mathbf{A}}\|_F$ entre todas las matrices de rango R. Este resultado, conocido como el teorema de Eckart-Young, garantiza que la información más relevante de la matriz se concentra en los primeros valores singulares [4].

En el contexto de las Tensor Networks, la SVD desempeña un doble papel. Por un lado, como se vio en la sección sobre entrelazamiento, la descomposición de Schmidt de un estado cuántico bipartito es formalmente una SVD del tensor de coeficientes reorganizado como matriz [10]. Por otro lado, la construcción algorítmica de las principales descomposiciones tensoriales –en particular la Tensor Train– procede mediante aplicaciones sucesivas de SVD, lo que hace de esta herramienta el enlace técnico entre el álgebra lineal clásica y las Tensor Networks [4].

Descomposición CP (CANDECOMP/PARAFAC).

La descomposición CP (*Canonical Polyadic*) es la generalización más directa del concepto de rango matricial a tensores de orden superior. Dado un tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1\times I_2\times\cdots\times I_N}$, la descomposición CP lo expresa como una suma de R tensores de rango 1:

$$\boldsymbol{\mathcal{X}} \approx \sum_{r=1}^{R} \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \cdots \circ \mathbf{a}_r^{(N)} \tag{1.19}$$

donde $\circ$ denota el producto exterior, cada $\mathbf{a}_r^{(n)} \in \mathbb{R}^{I_n}$ es un vector, y R es el *rango CP* del tensor, definido como el menor número de componentes de rango 1 necesarias para representarlo exactamente [14]. En componentes, la expresión anterior se escribe:

$$\mathcal{X}_{i_1 i_2 \ldots i_N} \approx \sum_{r=1}^{R} A^{(1)}{}_{i_1 r} A^{(2)}{}_{i_2 r} \cdots A^{(N)}{}_{i_N r} \tag{1.20}$$

donde $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$ son las *matrices de factores* cuyas columnas son los vectores $\mathbf{a}_r^{(n)}$ [4].

La descomposición CP tiene la ventaja de ofrecer una representación muy compacta: el número total de parámetros es $R \cdot \sum_{n=1}^{N} I_n$, que crece linealmente con el orden del tensor. Sin embargo, presenta una limitación importante: determinar el rango CP de un tensor dado es un problema NP-difícil, y los algoritmos de optimización (típicamente basados en mínimos cuadrados alternados) no garantizan la convergencia al óptimo global [4, 14]. En la notación diagramática, la descomposición CP se representa como un conjunto de nodos vectoriales conectados a un nodo central diagonal.

Descomposición de Tucker.

La descomposición de Tucker es una factorización más general que la CP. Dado un tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$, la descomposición de Tucker lo expresa como:

$$\mathcal{X}_{i_1 i_2 \ldots i_N} \approx \sum_{r_1=1}^{R_1} \sum_{r_2=1}^{R_2} \cdots \sum_{r_N=1}^{R_N} \mathcal{G}_{r_1 r_2 \ldots r_N} A^{(1)}{}_{i_1 r_1} A^{(2)}{}_{i_2 r_2} \cdots A^{(N)}{}_{i_N r_N} \tag{1.21}$$

donde $\boldsymbol{\mathcal{G}} \in \mathbb{R}^{R_1 \times R_2 \times \cdots \times R_N}$ es el *tensor núcleo* (*core tensor*) y cada $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$ es una matriz de factores. Los valores $R_1, R_2, \ldots, R_N$ se denominan *rangos de Tucker* y pueden ser diferentes entre sí, a diferencia del rango único R de la descomposición CP [4].

La descomposición de Tucker puede interpretarse como una transformación del tensor original a un espacio de dimensiones reducidas ($R_1 \times R_2 \times \cdots \times R_N$) a través de las matrices de factores, donde la estructura esencial del tensor queda capturada por el tensor núcleo $\boldsymbol{\mathcal{G}}$. Cuando se imponen restricciones de ortogonalidad sobre las matrices de factores, la descomposición de Tucker se denomina *Higher-Order Singular Value Decomposition* (HOSVD), que constituye la generalización natural de la SVD matricial al caso tensorial [4].

La relación entre las descomposiciones CP y Tucker es directa: la descomposición CP es un caso particular de la Tucker en el que el tensor núcleo $\boldsymbol{\mathcal{G}}$ es superdiagonal (es decir, sus elementos no nulos se encuentran únicamente en la diagonal principal generalizada) y todos los rangos son iguales: $R_1 = R_2 = \cdots = R_N = R$ [4].

El principal inconveniente de la descomposición de Tucker es que el tensor núcleo $\boldsymbol{\mathcal{G}}$ tiene $\prod_{n=1}^{N} R_n$ componentes, lo que sigue creciendo exponencialmente con el orden N si los rangos de Tucker son mayores que uno. Esto la hace impracticable para tensores de orden

muy alto, que es precisamente la situación habitual tanto en física cuántica de muchos cuerpos como en modelos de aprendizaje automático con un gran número de variables de entrada.

De las descomposiciones clásicas al Tensor Train.

Las limitaciones de las descomposiciones CP y Tucker motivaron la búsqueda de formatos que combinasen la compacidad de la CP con la flexibilidad de la Tucker, sin sufrir el crecimiento exponencial del tensor núcleo. El formato *Tensor Train* (TT), introducido por Oseledets en el ámbito de la matemática aplicada [15] y conocido como *Matrix Product State* (MPS) en física cuántica [10], resuelve este problema de forma elegante.

El formato TT descompone un tensor de orden N en una cadena de N tensores de orden 3 (salvo los extremos, que son de orden 2). La idea central es que, en lugar de un único tensor núcleo de tamaño $\prod_n R_n$ (como en Tucker), cada modo del tensor original se asocia a un tensor local $\boldsymbol{\mathcal{G}}^{(n)}$ de tamaño $R_{n-1} \times I_n \times R_n$, y los tensores se conectan secuencialmente mediante la contracción de sus índices virtuales.

Esta estructura tiene dos consecuencias fundamentales. En primer lugar, el número total de parámetros es $\sum_{n=1}^{N} R_{n-1} \cdot I_n \cdot R_n$, que crece linealmente con N para rangos fijos, en marcado contraste con el crecimiento exponencial de Tucker. En segundo lugar, la construcción del formato TT a partir de un tensor dado puede realizarse mediante $N-1$ descomposiciones SVD sucesivas, lo que garantiza estabilidad numérica y proporciona un control directo sobre el error de aproximación mediante el truncamiento de los valores singulares [4, 15].

La descomposición Tensor Train puede además entenderse como una secuencia de descomposiciones de Schmidt aplicadas iterativamente a cada bipartición del tensor, lo que cierra el círculo con la discusión del entrelazamiento cuántico de la sección anterior: los rangos TT R_n son exactamente los rangos de Schmidt de las biparticiones correspondientes, y su magnitud refleja el grado de correlación entre los primeros n modos y los restantes $N-n$ [10].

En la Tabla 1.1 se resumen las características principales de las tres descomposiciones presentadas.

Formato	Parámetros	Observaciones
CP	$R \cdot \sum_n I_n$	Compacta, pero el rango CP es NP-difícil de determinar.
Tucker	$\prod_n R_n + \sum_n I_n R_n$	Flexible, pero el núcleo crece exponencialmente con N.
TT / MPS	$\sum_n R_{n-1} I_n R_n$	Crece linealmente con N. Construible por SVD sucesivas.

Tabla 1.1: Comparativa de las descomposiciones tensoriales fundamentales para un tensor de orden N con dimensiones $I_1 \times I_2 \times \cdots \times I_N$.

1.2.5 Topologías de Tensor Networks

Las secciones anteriores han establecido que la eficacia de una Tensor Network para representar un tensor de alto orden depende de la estructura de correlaciones (o entrelazamiento) que se desea capturar. La topología de la red –es decir, la forma en que los tensores locales están conectados entre sí– determina qué patrones de correlación pueden representarse eficientemente y a qué coste computacional. En esta sección se describen en detalle las cuatro topologías más relevantes en la literatura.

Matrix Product State (MPS) / Tensor Train (TT).

El *Matrix Product State* (MPS), también denominado *Tensor Train* (TT) en matemática aplicada, es la topología de Tensor Network más estudiada y la que mejor se entiende desde un punto de vista teórico y algorítmico [10]. Su estructura consiste en una cadena lineal de N tensores de orden 3 conectados secuencialmente a través de índices virtuales de dimensión D (dimensión de enlace).

Desde el punto de vista físico, el MPS es la representación natural de los estados fundamentales de sistemas cuánticos unidimensionales con interacciones locales. Como se discutió en la sección sobre entrelazamiento, estos estados satisfacen la ley de área: el entrelazamiento entre cualquier bipartición contigua está acotado por una constante, lo que permite una representación fiel con dimensión de enlace finita [9].

Las correlaciones que un MPS puede representar decaen exponencialmente con la distancia entre sitios: la función de correlación entre los sitios i y j satisface $\langle O_i O_j \rangle - \langle O_i \rangle \langle O_j \rangle \sim e^{-|i-j|/\xi}$, donde ξ es una longitud de correlación que depende de la dimensión de enlace D [7]. Esto significa que el MPS es especialmente eficiente para capturar correlaciones locales o de corto alcance, pero puede requerir dimensiones de enlace elevadas para representar correlaciones de largo alcance.

Entre las ventajas del MPS destaca la existencia de algoritmos de optimización muy eficientes, heredados del DMRG. El coste computacional de estos algoritmos escala como $\mathcal{O}(ND^3d)$ para un solo barrido de optimización, lo que es polinomial en todos sus argu-

mentos [10]. El número total de parámetros del MPS es del orden de NdD^2, que crece linealmente con el tamaño del sistema N.

En el contexto del aprendizaje automático, el MPS ha sido la primera arquitectura tensorial empleada con éxito como clasificador de imágenes. Stoudenmire y Schwab demostraron que un MPS puede alcanzar una precisión inferior al 1 % de error en el conjunto MNIST, utilizando un algoritmo de optimización directamente inspirado en el DMRG [7]. Aunque el MPS impone una estructura de correlación unidimensional sobre los píxeles de la imagen –lo que a priori no se ajusta a la naturaleza bidimensional de las imágenes–, en la práctica la linearización de los píxeles resulta sorprendentemente efectiva para conjuntos de datos sencillos [7].

Tree Tensor Network (TTN).

El *Tree Tensor Network* (TTN) adopta una topología jerárquica en forma de árbol binario. En esta estructura, los tensores de la capa más baja (las hojas) están asociados a los índices físicos del tensor original, y se contraen por pares hacia tensores de capas superiores, reduciendo progresivamente el número de índices hasta llegar a un único tensor raíz que produce la salida [9].

Formalmente, en un TTN binario con $N = 2^L$ índices de entrada y L capas, cada tensor local tiene tres índices: dos de entrada (provenientes de la capa inferior) y uno de salida (conectado a la capa superior), con dimensión de enlace D. El número total de parámetros escala como $\mathcal{O}(ND^3)$, comparable al MPS, pero con una diferencia estructural crucial: la distancia entre cualquier par de índices físicos a través de la red es a lo sumo $\mathcal{O}(\log N)$ (la profundidad del árbol), frente a $\mathcal{O}(N)$ en el MPS [4].

Esta propiedad hace que el TTN sea más eficiente que el MPS para capturar correlaciones de largo alcance. En física cuántica, el TTN se utiliza para representar estados con una estructura de entrelazamiento jerárquica, como la que aparece en ciertos sistemas con simetría de escala. En aprendizaje automático, la estructura de árbol resulta natural para datos con organización jerárquica intrínseca [4].

Sin embargo, el TTN tiene una limitación: al ser una estructura de árbol sin ciclos, la información entre dos ramas distintas solo puede propagarse a través del nodo ancestro común. Esto limita la capacidad de la red para capturar correlaciones entre modos que se encuentran en ramas separadas del árbol. Esta restricción motivó el desarrollo de arquitecturas más sofisticadas como la MERA.

Multi-scale Entanglement Renormalization Ansatz (MERA).

La *Multi-scale Entanglement Renormalization Ansatz* (MERA) fue introducida por Vidal en 2007 como una Tensor Network capaz de representar eficientemente los estados fundamentales de sistemas cuánticos críticos, es decir, sistemas en los que las correlaciones decaen como una ley de potencias en lugar de exponencialmente [11].

La innovación clave de la MERA respecto al TTN es la inclusión de tensores adicionales denominados *disentanglers* (desentrelazadores). En cada capa de la estructura jerárquica,

antes de aplicar los tensores de renormalización (que reducen el número de grados de libertad, como en un TTN), se aplican disentanglers que actúan sobre pares de sitios vecinos para eliminar el entrelazamiento de corto alcance. Los tensores de renormalización se encargan entonces de capturar las correlaciones a escalas mayores. La alternancia de capas de disentanglers y capas de renormalización permite a la MERA representar correlaciones a todas las escalas de longitud simultáneamente [4].

Desde el punto de vista de la física, la MERA es la única topología de Tensor Network que puede representar fielmente estados con correcciones logarítmicas a la ley de área –la situación típica en sistemas cuánticos críticos–, donde el MPS requeriría una dimensión de enlace que crece polinomialmente con el tamaño del sistema [11]. La estructura jerárquica de la MERA está íntimamente relacionada con el concepto de grupo de renormalización, lo que le confiere una interpretación física muy rica: cada capa de la MERA corresponde a observar el sistema a una escala de longitud diferente.

En aprendizaje automático, la MERA resulta prometedora para el procesamiento de datos con dependencias a múltiples escalas, como imágenes con estructuras tanto locales como globales [4]. No obstante, su mayor complejidad estructural (más parámetros y contracciones más costosas que el MPS o el TTN) hace que su uso práctico sea más limitado hasta la fecha.

Projected Entangled Pair States (PEPS).

Los *Projected Entangled Pair States* (PEPS) constituyen la generalización natural del MPS a dos dimensiones. Mientras que en el MPS cada tensor se conecta a lo sumo con dos vecinos (formando una cadena), en PEPS cada tensor se conecta con cuatro vecinos, formando una red bidimensional que refleja directamente la geometría de una rejilla cuadrada [13].

Para un sistema bidimensional de $M \times N$ sitios, cada tensor local en PEPS tiene un índice físico de dimensión d y cuatro índices virtuales (arriba, abajo, izquierda, derecha) de dimensión D. Esto permite capturar de forma natural las correlaciones espaciales bidimensionales que caracterizan, por ejemplo, los estados fundamentales de sistemas cuánticos definidos sobre rejillas planas [4].

PEPS satisface la ley de área de forma nativa en dos dimensiones: la entropía de entrelazamiento entre una región A y su complemento escala con el perímetro de A, exactamente como predicen los resultados teóricos para estados de baja energía en sistemas bidimensionales [9]. Esto convierte a PEPS en la arquitectura tensorial teóricamente más adecuada para datos con estructura espacial bidimensional, como las imágenes.

Sin embargo, PEPS presenta una desventaja computacional significativa: la contracción exacta de una red PEPS es un problema computacionalmente intratable (#P-difícil), a diferencia del MPS, donde la contracción exacta tiene un coste polinomial. En la práctica, esto obliga a utilizar métodos de contracción aproximada, lo que introduce errores adicionales y limita el tamaño de las dimensiones de enlace que pueden utilizarse [4]. A pesar de esta limitación, PEPS ha demostrado resultados competitivos en clasificación de imágenes, alcanzando rendimientos comparables a los de arquitecturas clásicas como

GoogLeNet, VGG-16 y AlexNet [2].

Comparativa y selección de topología.

La elección de la topología adecuada para una tarea concreta depende de dos factores: la estructura de correlaciones que se desea capturar y las restricciones computacionales disponibles. En la Tabla 1.2 se resumen las características principales de las cuatro topologías descritas.

Topología	Parámetros	Correlaciones	Aplicación principal
MPS / TT	$\mathcal{O}(NdD^2)$	Exponenciales	Sistemas 1D; clasificación con datos linearizados.
TTN	$\mathcal{O}(ND^3)$	Jerárquicas	Datos con estructura multiescala; sistemas jerárquicos.
MERA	$\mathcal{O}(ND^4)$	Ley de potencias	Sistemas críticos; datos con correlaciones a todas las escalas.
PEPS	$\mathcal{O}(NdD^4)$	2D nativas	Sistemas 2D; imágenes con estructura espacial.

Tabla 1.2: Comparativa de las principales topologías de Tensor Networks. N es el número de sitios, D la dimensión de enlace y d la dimensión local.

En el contexto de este trabajo, las arquitecturas de mayor interés son el MPS/TT –por su simplicidad, su conexión directa con el DMRG y la existencia de implementaciones maduras– y las redes jerárquicas (TTN y DTTN), que han demostrado recientemente un rendimiento competitivo en tareas de clasificación de imágenes complejas [4].

1.3 Clasificación de imágenes con Tensor Networks

Las dos secciones anteriores han presentado, por un lado, los fundamentos del Deep Learning y, por otro, la teoría de Tensor Networks desde su origen en la física cuántica hasta sus principales topologías y descomposiciones. A partir de aquí, el siguiente paso natural es unir ambos marcos. Esta sección describe, por tanto, el procedimiento completo mediante el cual una Tensor Network puede utilizarse como modelo de clasificación supervisada de imágenes, siguiendo el enfoque propuesto por Stoudenmire y Schwab [7].

1.3.1 Codificación de los datos de entrada

El primer paso para aplicar una Tensor Network a la clasificación de imágenes consiste en transformar los datos de entrada –los píxeles de una imagen– en un objeto tensorial con la estructura adecuada. Esta transformación se denomina *mapa de características* (*feature map*) y su diseño está directamente inspirado en la mecánica cuántica.

Consideremos una imagen en escala de grises con N píxeles, donde el valor de cada píxel $x_j \in [0,1]$ (0 para blanco, 1 para negro). El procedimiento consiste en dos pasos. En primer lugar, se aplica un *mapa de características local* $\phi^{s_j}(x_j)$ que transforma cada píxel individual en un vector de dimensión d, denominada *dimensión local*. En segundo lugar, se construye el mapa de características global $\Phi(\mathbf{x})$ como el producto tensorial de todos los mapas locales:

$$\Phi^{s_1 s_2 \cdots s_N}(\mathbf{x}) = \phi^{s_1}(x_1) \otimes \phi^{s_2}(x_2) \otimes \cdots \otimes \phi^{s_N}(x_N) \tag{1.22}$$

El tensor $\Phi(\mathbf{x})$ resultante es un tensor de orden N con d^N componentes, pero posee una estructura muy particular: es un tensor de rango 1 (producto de N vectores). En el lenguaje de la mecánica cuántica, $\Phi(\mathbf{x})$ tiene la estructura de un *estado producto* o función de onda no entrelazada [7].

La elección concreta del mapa local $\phi^{s_j}(x_j)$ determina las propiedades del modelo. Para el caso $d = 2$, Stoudenmire y Schwab propusieron el mapa:

$$\phi^{s_j}(x_j) = \left[\cos\left(\frac{\pi}{2}x_j\right),\ \sin\left(\frac{\pi}{2}x_j\right)\right] \tag{1.23}$$

que tiene una interpretación física elegante: cada píxel se codifica como el estado normalizado de un *qubit*, donde el estado $|0\rangle$ ("espín arriba") corresponde a un píxel blanco, el estado $|1\rangle$ ("espín abajo") a un píxel negro, y cualquier valor intermedio de gris se representa como una superposición cuántica de ambos estados [7]. La elección trigonométrica garantiza que $\|\phi(x_j)\| = 1$ para todo x_j, lo que implica que $\|\Phi(\mathbf{x})\| = 1$ para cualquier imagen. Esta normalización es análoga a la exigencia de normalización de las funciones de onda en mecánica cuántica.

Para dimensiones locales $d > 2$, el mapa puede generalizarse utilizando estados coherentes de espín, lo que permite capturar variaciones más finas en los valores de los píxeles a costa de un mayor número de parámetros [7].

1.3.2 Función de decisión y clasificación multiclase

Una vez construida esta representación tensorial de la imagen, el siguiente paso consiste en definir cómo se lleva a cabo la clasificación. Para una tarea con N_L etiquetas posibles $\ell \in \{1, 2, \ldots, N_L\}$, se introduce un tensor de pesos W^ℓ de orden $N+1$ (los N índices del mapa de características más un índice de etiqueta ℓ), y la función de decisión viene dada por:

$$f^\ell(\mathbf{x}) = W^\ell \cdot \Phi(\mathbf{x}) = \sum_{s_1, s_2, \ldots, s_N} W^\ell_{s_1 s_2 \cdots s_N}\, \phi^{s_1}(x_1)\, \phi^{s_2}(x_2) \cdots \phi^{s_N}(x_N) \tag{1.24}$$

La imagen $\mathbf{x}$ se clasifica con la etiqueta ℓ^* para la cual $|f^\ell(\mathbf{x})|$ es máximo [7].

Es importante destacar la naturaleza de esta operación: aunque la función de decisión f^ℓ es lineal en el espacio de características d^N-dimensional, es altamente no lineal en el espacio de entrada original (el de los píxeles). En efecto, la función $f^\ell(\mathbf{x})$ es un polinomio de grado N en las funciones trigonométricas de los valores de los píxeles. Esta es precisamente la estrategia que emplean las Tensor Networks para lograr separabilidad lineal sin recurrir a funciones de activación no lineales, a diferencia de las redes neuronales convencionales [4].

El tensor de pesos W^ℓ tiene, en principio, $N_L \cdot d^N$ componentes –un número exponencialmente grande–. Para hacerlo tratable, se representa como una Tensor Network. En el enfoque de Stoudenmire y Schwab, W^ℓ se aproxima como un MPS:

$$W^\ell_{s_1 s_2 \cdots s_N} = \sum_{\{\alpha\}} A^{\alpha_1}_{s_1} A^{\alpha_1\alpha_2}_{s_2} \cdots A^{\ell;\alpha_j\alpha_{j+1}}_{s_j} \cdots A^{\alpha_{N-1}}_{s_N} \tag{1.25}$$

donde el índice de etiqueta ℓ se coloca en uno de los tensores del MPS (su posición es arbitraria y puede moverse a cualquier sitio mediante operaciones de SVD). La dimensión de enlace D del MPS controla la capacidad del modelo: un D demasiado pequeño puede resultar en *underfitting*, mientras que un D demasiado grande puede provocar *overfitting* [7].

1.3.3 Optimización del modelo: el algoritmo de barrido

Una vez definida la función de decisión y fijada la estructura MPS para el tensor de pesos, queda por explicar cómo se ajustan sus parámetros. La optimización se realiza mediante un algoritmo de barrido (*sweeping algorithm*) directamente inspirado en el DMRG de la física cuántica. El procedimiento minimiza una función de coste cuadrática:

$$C = \frac{1}{2} \sum_{n=1}^{N_T} \sum_{\ell} \left(f^\ell(\mathbf{x}_n) - \delta^\ell_{L_n} \right)^2 \tag{1.26}$$

donde N_T es el número de muestras de entrenamiento, $\mathbf{x}_n$ es la n-ésima imagen, L_n es su etiqueta correcta, y $\delta^\ell_{L_n}$ es la delta de Kronecker (igual a 1 si $\ell = L_n$ y 0 en caso contrario) [7].

El algoritmo procede de la siguiente manera. En cada paso, se seleccionan dos tensores vecinos del MPS en los sitios j y $j+1$, y se contraen para formar un único *tensor de enlace* B^ℓ. A continuación, se calcula el gradiente de la función de coste respecto a B^ℓ y se actualiza este tensor mediante un paso de descenso de gradiente. Tras la actualización, el tensor B^ℓ se descompone de nuevo en dos tensores separados mediante una SVD, restaurando la forma MPS. En este paso, solo se conservan los D valores singulares más grandes, lo que permite controlar adaptativamente la dimensión de enlace [7].

Este procedimiento se repite "barriendo" de izquierda a derecha y de derecha a izquierda

a lo largo de toda la cadena MPS, de forma análoga a como el DMRG barre la cadena de espines en un problema de muchos cuerpos. La convergencia es típicamente rápida: Stoudenmire y Schwab reportaron que dos o tres barridos completos eran suficientes para alcanzar una buena convergencia en el conjunto MNIST, logrando un error de test inferior al 1 % con una dimensión de enlace de $D = 120$ [7].

Entre las propiedades destacables del algoritmo se encuentran: el coste computacional escala linealmente con el tamaño de la entrada N y con el número de muestras de entrenamiento N_T; la dimensión de enlace puede adaptarse localmente en cada enlace del MPS, concentrando recursos en las regiones del modelo que más lo necesitan; y la truncación por SVD proporciona una regularización implícita del modelo, equivalente a imponer una restricción de bajo rango sobre el tensor de pesos [7].

Un aspecto práctico relevante es la linearización de los píxeles. Dado que el MPS impone una estructura unidimensional, los píxeles de la imagen bidimensional deben ordenarse en una secuencia lineal. Stoudenmire y Schwab emplearon un orden en zigzag que mantiene la proximidad espacial entre píxeles vecinos en la medida de lo posible [7]. La sensibilidad del modelo al orden de linearización es una de las motivaciones para explorar topologías bidimensionales como PEPS o jerárquicas como TTN y DTTN.

1.3.4 Conexión con métodos kernel

Además de esta lectura en términos de redes tensoriales, el marco descrito admite una interpretación alternativa dentro de la teoría de aprendizaje con *kernels*, que proporciona una perspectiva complementaria sobre el funcionamiento de estos clasificadores.

En el aprendizaje con kernels, la clasificación se realiza en un espacio de características de alta dimensión sin construir explícitamente la representación de los datos en dicho espacio, sino a través de una función kernel $K(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x}) \cdot \Phi(\mathbf{x}')$ que mide la similitud entre dos entradas. El mapa de características tensorial de la Ecuación 1.22 define implícitamente un kernel que es el producto de N kernels locales, uno por cada componente de la entrada:

$$K(\mathbf{x}, \mathbf{x}') = \prod_{j=1}^{N} \sum_{s_j} \phi^{s_j}(x_j)\, \phi^{s_j}(x'_j) = \prod_{j=1}^{N} k(x_j, x'_j) \tag{1.27}$$

donde $k(x_j, x'_j) = \cos\left(\frac{\pi}{2}(x_j - x'_j)\right)$ para el mapa local de la Ecuación 1.23. Kernels de este tipo –productos de kernels locales independientes– son adecuados para datos en los que no se asume ninguna relación previa entre las distintas componentes del vector de entrada antes del aprendizaje [7].

La diferencia fundamental entre un clasificador kernel clásico (como una máquina de vectores de soporte) y el enfoque tensorial es cómo se representan y optimizan los pesos. En un clasificador kernel, los pesos se expresan como combinación lineal de los vectores de entrenamiento y el coste de evaluación crece con el número de muestras. En el enfoque ten-

sorial, los pesos se representan como un MPS, cuyo coste de evaluación es independiente del número de muestras y depende únicamente de la dimensión de enlace D [7].

Además, la estructura MPS del tensor de pesos proporciona una selección implícita de características. Stoudenmire y Schwab demostraron que un MPS con tensor central $C^{\ell}_{\alpha_i \alpha_{i+1}}$ en un enlace i puede interpretarse como un modelo que actúa en tres etapas: primero, la entrada se mapea al espacio de características exponencial $\Phi(\mathbf{x})$; después, las matrices unitarias U y V del MPS proyectan este espacio exponencial a un subespacio de dimensión D^2; y finalmente, la clasificación se realiza en este subespacio reducido [7]. Esta interpretación revela que el MPS no solo parametriza eficientemente el tensor de pesos, sino que simultáneamente aprende a seleccionar las características más relevantes para la tarea de clasificación, de forma análoga a como las capas ocultas de una red neuronal aprenden representaciones internas de los datos.

Capítulo 2. OBJETIVOS

2.1 Objetivos generales

El objetivo general del trabajo consiste en ampliar el conocimiento actual sobre la aplicación de las Tensor Networks como paradigma alternativo al deep learning convencional en tareas de clasificación de imágenes, evaluando su viabilidad, rendimiento y eficiencia mediante la implementación y experimentación con distintas arquitecturas tensoriales.

Para ello, el proyecto contempla el estudio en profundidad del estado del arte en modelos de inteligencia artificial basados en Tensor Networks, la implementación de al menos dos arquitecturas tensoriales a partir de la literatura científica, su entrenamiento y evaluación utilizando recursos de computación de altas prestaciones (HPC) del servidor Lorca de la Universidad Europea, y la comparación de sus resultados frente a modelos clásicos de deep learning sobre conjuntos de datos de referencia.

De forma complementaria, el proyecto persigue adquirir experiencia práctica en el uso de herramientas computacionales propias del ámbito de la investigación en inteligencia artificial, incluyendo el manejo de entornos HPC, el uso de librerías especializadas como PyTorch y TensorKrowch, y las buenas prácticas de documentación y reproducibilidad del código científico.

2.2 Objetivos específicos

1. Realizar una revisión del estado del arte de los modelos de inteligencia artificial basados en Tensor Networks aplicados a la clasificación de imágenes, analizando las arquitecturas más relevantes y sus principales resultados.

2. Estudiar en detalle distintos modelos tensoriales para clasificación de imágenes, con el fin de comprender su funcionamiento y sus diferencias fundamentales.

3. Implementar al menos dos modelos de inteligencia artificial basados en Tensor Networks para tareas de clasificación de imágenes, a partir de descripciones presentes en la literatura científica.

4. Entrenar y evaluar los modelos implementados utilizando recursos de computación de altas prestaciones (HPC), familiarizándose con el uso de servidores, entornos de ejecución y herramientas básicas de gestión de trabajos.

5. Comparar el rendimiento de los modelos basados en Tensor Networks con modelos de inteligencia artificial convencionales, empleando conjuntos de datos de referencia en clasificación de imágenes.

6. Analizar los resultados obtenidos en términos de precisión, eficiencia computacional y comportamiento del entrenamiento, identificando las principales ventajas y limitaciones de los enfoques tensoriales.

7. Documentar el desarrollo del proyecto, describiendo de forma clara los modelos estudiados, las implementaciones realizadas y los resultados obtenidos.

8. Preparar y organizar un repositorio de código que recoja las implementaciones y experimentos realizados, garantizando su claridad, reproducibilidad y correcta documentación.

2.3 Objetivos que quedan fuera del proyecto

- Estudiar modelos de inteligencia artificial no aplicados a la clasificación de imágenes y que no estén basados en Tensor Networks.
- Evaluar los modelos en conjuntos de datos distintos de los considerados en el proyecto, o realizar comparaciones extensivas en múltiples benchmarks adicionales.
- Ejecutar los modelos en hardware cuántico real, más allá de enfoques cuántico-inspirados ejecutados en sistemas clásicos.
- Desarrollar aplicaciones orientadas a entornos industriales o sistemas en producción a partir de los modelos estudiados.

2.4 Beneficios del proyecto

Desde el punto de vista científico, este proyecto contribuye a ampliar el conocimiento empírico sobre las Tensor Networks como herramienta de aprendizaje automático. Si bien el fundamento teórico de estas arquitecturas está bien establecido en el ámbito de la física cuántica y matemática, su comportamiento práctico en tareas de clasificación de imágenes sigue siendo un área activa de investigación. El trabajo aportará evidencia experimental concreta sobre estas cuestiones.

Desde el punto de vista computacional, el proyecto favorece el desarrollo de modelos de IA más eficientes en el uso de parámetros. Los modelos basados en Tensor Networks permiten, en principio, representar funciones complejas con un número significativamente menor de parámetros que las redes neuronales profundas convencionales. Esto tiene implicaciones directas sobre el coste energético y computacional del entrenamiento, un aspecto de creciente relevancia en la investigación en inteligencia artificial.

Desde el punto de vista formativo, el proyecto supone una experiencia integral en investigación aplicada: comprende la revisión crítica de literatura científica, el diseño e implementación de experimentos reproducibles, el uso de infraestructuras HPC y el aprendizaje de herramientas y flujos de trabajo propios del ámbito profesional de la investigación en IA.

Estos conocimientos y competencias son directamente transferibles a entornos académicos y laborales en el campo de la inteligencia artificial y la computación científica

Capítulo 3. DESARROLLO DEL TFG

3.1 Planificación del trabajo fin de grado

Figura 3.1: Planificación por semanas del TFG, cada circulo representa una semana teniendo febrero 3 semanas (porque empiezo el 9 de Febrero), marzo 4 semanas, abril cuatro semanas y por último mayo tendrá otras cuatro semanas. De esta manera se cumplen entre 500 y 600 horas dedicadas al TFG.

- **Semanas 1–3: Introducción y fundamentos de Tensor Networks**

  Durante las primeras semanas se adquirirán los conocimientos fundamentales y teóricos sobre Tensor Networks.

  - Estudio de la notación y representaciones básicas de Tensor Networks mediante material audiovisual.

- Lectura y análisis de artículos científicos relevantes, con especial atención a modelos tensoriales aplicados a clasificación de imágenes.
- Estudio de apuntes y material teórico relacionado con Tensor Networks y su conexión con la física de sistemas cuánticos.

- **Semanas 4–6: Estudio de modelos MPS para clasificación de imágenes**

  En esta etapa se profundizará en el uso de Matrix Product States como modelos de clasificación de imágenes:

  - Análisis del funcionamiento de clasificadores basados en MPS y estudio de sus fuentes de error.
  - Revisión de implementaciones existentes y comprensión de los procedimientos de entrenamiento y evaluación.
  - Búsqueda y selección de conjuntos de datos adecuados en plataformas como Kaggle o Hugging Face.

- **Semanas 7–8: Formación en herramientas computacionales**

  Durante este periodo se adquirirá el conocimiento práctico necesario para el desarrollo del proyecto:

  - Aprendizaje de comandos básicos de Bash y trabajo en entornos de servidor.
  - Uso de GitHub para el control de versiones y la organización del código.
  - Estudio de buenas prácticas de organización de proyectos y documentación.
  - Aprendizaje en profundidad de PyTorch para la implementación de modelos de inteligencia artificial.

- **Semanas 9–13: Implementación y experimentación**

  En esta fase se llevará a cabo la implementación y entrenamiento de los modelos seleccionados:

  - Implementación de modelos basados en Tensor Networks para clasificación de imágenes.
  - Entrenamiento de los modelos utilizando recursos de computación de altas prestaciones (HPC).
  - Evaluación del rendimiento de los modelos y comparación con modelos de inteligencia artificial convencionales.

- **Semanas 13–15: Análisis de resultados y redacción**

  En la etapa final se realizará el análisis de los resultados obtenidos y la documentación del trabajo:

  - Análisis crítico de los resultados y discusión de las conclusiones.
  - Redacción de la memoria del Trabajo Fin de Grado.
  - Organización final del repositorio de código y preparación de la entrega.

3.2 Descripción de la solución, metodologías y herramientas empleadas

3.2.1 Recursos requeridos

A continuación se enumeran los recursos técnicos utilizados para la ejecución del trabajo:

- Recursos computacionales:
  - Servidor HPC Lorca de la Universidad Europea de Madrid: para el entrenamiento de los modelos con recursos de computación de altas prestaciones.
  - Ordenador personal: para el desarrollo, pruebas locales y escritura de la memoria.
- Herramientas de software
  - Python (lenguaje de programación principal).
  - PyTorch (framework de deep learning).
  - TensorKrowch (librería para implementación de Tensor Networks).
  - NumPy, SciPy, Matplotlib, Seaborn (librerías científicas y de visualización).
  - GitHub (control de versiones y gestión del repositorio de código).
  - Bash / Linux (gestión del entorno HPC y automatización de trabajos).
  - Docker / entornos virtuales (gestión de dependencias y reproducibilidad).
- Recursos bibliográficos y de datos:
  - Artículos científicos de referencia (véase sección de Referencias).
  - Conjuntos de datos públicos: MNIST, OlivettiFaces (obtenidos de repositorios como Kaggle o Hugging Face).

3.3 Resultados del TFG

Conforme a los objetivos específicos, describe los resultados finales obtenidos.

Puedes añadir una descripción de cambios durante el TFG respecto a los objetivos iniciales, y comentarios que quieras incluir de cómo has desarrollado las distintas actividades.

Capítulo 4. DISCUSIÓN

Esta sección es más habitual en trabajos de tipo "científico" o de "investigación", donde uno presenta brevemente los resultados principales y los discute, pero también puede utilizarse en otro tipo de trabajos.

Puedes incluir secciones específicas para discutir cuestiones como: limitaciones del estudio, limitaciones de la tecnología empleada, cambios respecto a objetivos planteados inicialmente.

Puedes incluir respuestas a preguntas del tipo: ¿la metodología inicialmente pensada ha sido útil?, ¿he tenido que adaptarme a cambios a lo largo del proyecto?, ¿qué cambios han sido y cómo he adaptado el proyecto para poder manejar esos cambios?, ¿qué impacto ha tenido el resultado de mi proyecto?

Capítulo 5. CONCLUSIONES

5.1 Conclusiones del trabajo

Breve descripción objetiva del resultado en relación al objetivo general de tu TFG.

5.2 Conclusiones personales

Describe tus impresiones y experiencia personal durante el desarrollo del TFG, o destacar la importancia que tiene el tema para ti, lo que has aprendido, o la trascendencia que ha tenido para ti o para otros este trabajo.

Capítulo 6. FUTURAS LÍNEAS DE TRABAJO

Es importante destacar en este apartado las líneas con las que se podría continuar tu trabajo. Indica todo lo que has anotado como futuro trabajos durante el desarrollo del TFG, o aquellos aspectos que estaban fuera del alcance, pero que son interesantes para desarrollar a futuro el valor de los resultados de tu trabajo.

REFERENCIAS

[1] Fred Van Veen and Stefan Leijnen. The neural network zoo. `https://www.asimovinstitute.org/neural-network-zoo`, 2019. Asimov Institute. Accessed: 2026-03-19.

[2] Alejandro Mata Ali, Iñigo Perez Delgado, and Aitor Moreno Fdez. de Leceta. Quantum-inspired Techniques in Tensor Networks for Industrial Contexts. *arXiv e-prints*, page arXiv:2404.11277, April 2024.

[3] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.

[4] Maolin Wang, Yu Pan, Zenglin Xu, Guangxi Li, Xiangli Yang, Danilo Mandic, and Andrzej Cichocki. Tensor networks meet neural networks: A survey and future perspectives. *arXiv preprint arXiv:2302.09019*, 2023.

[5] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P Mandic. Tensor networks for dimensionality reduction and large-scale optimization part 1 low-rank tensor decompositions. *Foundations and Trends in Machine Learning*, 9(4-5):249–429, 2016.

[6] Roger Penrose et al. Applications of negative dimensional tensors. *Combinatorial mathematics and its applications*, 1(221-244):3, 1971.

[7] E. Miles Stoudenmire and David J. Schwab. Supervised learning with quantum-inspired tensor networks, 2017.

[8] Jacob Biamonte and Ville Bergholm. Tensor networks in a nutshell, 2017.

[9] Amandeep Singh Bhatia, Mandeep Kaur Saggi, and Sabre Kais. Application of quantum-inspired tensor networks to optimize federated learning systems. *Quantum Machine Intelligence*, 7(1):12, 2025.

[10] Ulrich Schollwoeck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics - ANN PHYS N Y*, 326, 08 2010.

[11] G. Vidal. Entanglement renormalization. *Phys. Rev. Lett.*, 99:220405, Nov 2007.

[12] Steven R. White. Density matrix formulation for quantum renormalization groups. *Phys. Rev. Lett.*, 69:2863–2866, Nov 1992.

[13] F. Verstraete and J. I. Cirac. Renormalization algorithms for quantum-many body systems in two and higher dimensions. *arXiv preprint cond-mat/0407066*, 2004.

[14] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.

[15] Ivan Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33:2295–2317, 09 2011.

ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).

[PÁGINA INTENCIONADAMENTE EN BLANCO]